

Multi – level Linked List
–Modified Insert After Element
–Growth Rate of Output

Program:

```
Node *searchNode(Node *current, int target)
{
    if (current == nullptr)
        return nullptr;

    if (current->data == target)
        return current;

    Node *foundInChild = searchNode(current->child, target);
    if (foundInChild != nullptr)
    {
        return foundInChild;
    }
    return searchNode(current->next, target);
}

void insertAfterElement()
{
    if (head == nullptr)
    {
```

```
        cout << "List is empty. Please insert an element  
first." << endl;  
        return;  
    }  
  
    int target, data, type;  
    cout << "Enter the element to search for: ";  
    cin >> target;  
  
    Node *temp = searchNode(head, target);  
  
    if (temp == nullptr)  
    {  
        cout << "Element " << target << " not found anywhere  
in the list." << endl;  
        return;  
    }  
  
    cout << "Enter data to insert: ";  
    cin >> data;  
    Node *newNode = createNode(data);  
  
    cout << "Insert as (1) Next Node or (2) Child Node? ";  
    cin >> type;  
  
    if (type == 1)  
    {
```

```

        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted " << data << " as NEXT of " <<
target << endl;
    }
    else if (type == 2)
    {
        newNode->next = temp->child;
        temp->child = newNode;
        cout << "Inserted " << data << " as CHILD of " <<
target << endl;
    }
    else
    {
        cout << "Invalid option." << endl;
        free(newNode);
    }
}
int main()
{
    int choice;
    while (true)
    {
        cout << "4. Insert After Element (Link or Child)" <<
endl;
        switch (choice)
        {
            case 4:

```

```

        insertAfterElement();
    default:
        cout << "Invalid choice" << endl;
    }
}
return 0;
}

```

Growth Rate of Output of `searchNode(current, target)`

Let

$P(n)$ = number of nodes successfully examined (visited) while searching for the target.

Each visited node contributes exactly one comparison

```

if (current->data == target)

```

and is visited exactly once.

Suppose at some node

- *c = number of nodes in its child subtree*
- *r = number of nodes reachable through its next pointer*
- *1 node is the current node,*

The search proceeds as follows:

- 1. Examine the current node.*
- 2. Search the child subtree.*
- 3. If not found, search the next subtree.*

Hence,

$$n = c + r + 1$$

```
if (current->data == target)
```

The current node is examined once.

Contribution:

$$+1$$

```
Node *foundInChild = searchNode(current->child, target);
```

If the child subtree contains c nodes, then the search inside that subtree contributes.

$$P(c)$$

visited nodes.

(3) Search the next subtree

If the target is not found in the child subtree,

```
return searchNode(current->next, target);
```

the remaining r nodes are searched.

Contribution:

$$P(r)$$

Step 4. Combine the three parts

Therefore,

$$P(n) = P(c) + P(r) + 1$$

Since

$$n = c + r + 1,$$

assume by induction that

$$P(c) = c, P(r) = r.$$

Then

$$\begin{aligned} P(n) &= P(c) + P(r) + 1 \\ &= c + r + 1 \\ &= n. \end{aligned}$$

Therefore,

$$P(n) = n = \Theta(n)$$

And this $\Theta(n)$ is during worst case.

For Best Case

*If the target is the very first node (head),
only one node is examined.*

$$P(n) = \Theta(1)$$

For Average Case

On average, the target is found after examining about half the nodes.

Let

$P_{avg}(n)$ = *expected number of nodes examined.*

Suppose the traversal order is:

$$v_1, v_2, v_3, \dots, v_n.$$

If the target is

- v_1 , we examine 1 node.
- v_2 , we examine 2 nodes.
- v_3 , we examine 3 nodes.
- ...
- v_n , we examine n nodes.

Since every node is equally likely,

$$\Pr(v_i) = \frac{1}{n}. [\Pr(v_i) \text{ is Probability of } v_i]$$

Therefore,

$$P_{avg}(n) = \frac{1}{n} (1 + 2 + 3 + \dots + n)$$

Using,

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

We obtain,

$$P_{avg}(n) = \frac{1}{n} \times \frac{n(n+1)}{2} = \frac{n+1}{2} = \Theta(n)$$

Growth Rate

$$P_{avg}(n) = \frac{1}{n} \times \sum_{i=1}^n i = \frac{n+1}{2} = \Theta(n)$$

Final Growth Rate of Search Output

<i>Case</i>	<i>Nodes Examined</i>	<i>Growth Rate</i>
<i>Best Case</i>	1	$\Theta(1)$
<i>Average Case</i>	$\frac{n}{2}$	$\Theta(n)$
<i>Worst Case</i>	n	$\Theta(n)$

Growth Rate of Output of `insertAfterElement()`

Now define,

$I(n)$ = number of status messages printed by `insertAfterElement()`.

The function prints only

- List is empty
- Element not found
- Inserted...
- *prompts for input*

These are all constant.

Therefore,

$$I(n) = \Theta(1)$$

Total Growth Rate (Search + Insert)

<i>Case</i>	<i>Search $S(n)$</i>	<i>Insertion</i>	<i>Total $P(n)$</i>	<i>Growth</i>
<i>Best</i>	1	1	2	$\theta(1)$
<i>Average</i>	$\frac{n+1}{2}$	1	$\frac{n+3}{2}$	$\theta(n)$
<i>Worst</i>	n	1	$n+1$	$\theta(n)$
